Topic 11:

```
import Foundation

struct Stack<Element> {
    private var items: [Element] = []

    mutating func push(_ item: Element) {
        items.append(item)
    }

    mutating func pop() -> Element? {
        return items.popLast()
    }

    var top: Element? {
        return items.last
    }

    var isEmpty: Bool {
        return items.isEmpty
    }

    var count: Int {
        return items.count
    }
}

var intStack = Stack<Int>()
intStack.push(10)
intStack.push(20)
intStack.push(30)

print("Int top:", intStack.top ?? 0)
print("Int count:", intStack.count)
print("Int pop:", intStack.pop() ?? 0)

var stringStack = Stack<String>()
stringStack.push("Apple")
stringStack.push("Banana")
stringStack.push("Orange")

print("String top:", stringStack.top ?? "")
print("String count:", stringStack.count)
print("String pop:", stringStack.pop() ?? "")

func largest<T: Comparable>(in items: [T]) -> T? {
    guard var largest = items.first else {
        return nil
    }

    for item in items {
        if item > largest {
            largest = item
        }
    }

    return largest
}
```

```
print("Largest Int:", largest(in: [10, 50, 30, 80, 20]) ?? 0)
print("Largest String:", largest(in: ["Apple", "Zebra", "Mango"]) ?? "")

func swapValues<T>(_ a: inout T, _ b: inout T) {
    let temp = a
    a = b
    b = temp
}

var a = 10
var b = 20

print("Before swap:", a, b)

swapValues(&a, &b)

print("After swap:", a, b)
```

Output

```
Int top: 30
Int count: 3
Int pop: 30
String top: Orange
String count: 3
String pop: Orange
Largest Int: 80
Largest String: Zebra
Before swap: 10 20
After swap: 20 10
```

2.
```
enum ATMError: Error {
    case insufficientFunds(shortBy: Double)
    case invalidAmount
    case dailyLimitExceeded
}

struct Account {
    var balance: Double

    mutating func withdraw(_ amount: Double) throws {
        if amount <= 0 {
            throw ATMError.invalidAmount
        }

        if amount > 20000 {
            throw ATMError.dailyLimitExceeded
        }

        if amount > balance {
            throw ATMError.insufficientFunds(
                shortBy: amount - balance
            )
        }
```

```
        balance -= amount
        print("Withdrawal successful")
        print("Balance: \(balance)")
    }
}

var account = Account(balance: 1000)

// 1. Withdraw 500
do {
    try account.withdraw(500)
} catch ATMError.insufficientFunds(let shortBy) {
    print("Insufficient funds: \(shortBy)")
} catch ATMError.invalidAmount {
    print("Invalid amount")
} catch ATMError.dailyLimitExceeded {
    print("Daily limit exceeded")
} catch {
    print("Unknown error")
}

do {
    try account.withdraw(-10)
} catch ATMError.insufficientFunds(let shortBy) {
    print("Insufficient funds: \(shortBy)")
} catch ATMError.invalidAmount {
    print("Invalid amount")
} catch ATMError.dailyLimitExceeded {
    print("Daily limit exceeded")
} catch {
    print("Unknown error")
}

do {
    try account.withdraw(25000)
} catch ATMError.insufficientFunds(let shortBy) {
    print("Insufficient funds: \(shortBy)")
} catch ATMError.invalidAmount {
    print("Invalid amount")
} catch ATMError.dailyLimitExceeded {
    print("Daily limit exceeded")
} catch {
    print("Unknown error")
}


do {
    try account.withdraw(2000)
} catch ATMError.insufficientFunds(let shortBy) {
    print("Insufficient funds: \(shortBy)")
} catch ATMError.invalidAmount {
    print("Invalid amount")
} catch ATMError.dailyLimitExceeded {
    print("Daily limit exceeded")
} catch {
    print("Unknown error")
}

let result = try? account.withdraw(100)
```

```
if result != nil {
   print("Withdrawal worked")
} else {
   print("Withdrawal failed")
}
```

```
Withdrawal successful
Balance: 500.0
Invalid amount
Daily limit exceeded
Insufficient funds: 1500.0
Withdrawal successful
Balance: 400.0
Withdrawal worked
```

3.
```
enum AgeError: Error {
   case empty
   case notANumber
   case outOfRange
}

func parseAge(_ text: String) throws -> Int {
   if text.isEmpty {
      throw AgeError.empty
   }

   guard let age = Int(text) else {
      throw AgeError.notANumber
   }

   if age < 1 || age > 120 {
      throw AgeError.outOfRange
   }

   return age
}

let result = Result {
   try parseAge("25")
}

switch result {
case .success(let age):
   print("Valid age: \(age)")

case .failure(let error):
   switch error {
   case AgeError.empty:
      print("Error: Empty input")
   case AgeError.notANumber:
      print("Error: Not a number")
   case AgeError.outOfRange:
      print("Error: Age must be 1 to 120")
```

```
    default:
        print("Unknown error")
    }
}

enum ReportError: Error {
    case failed
}

func saveReport() throws {
    print("Opening file")

    defer {
        print("Closing file")
    }

    print("Saving report")
    throw ReportError.failed
}

do {
    try saveReport()
} catch {
    print("Report saving failed")
}
```

